

Implementación Patrón Template y State



Presentado por:

Brayan Stiven Charry Vela
Jhoiner Alberto Puentes Figueroa
Sebastian Ruiz Segura

Profesor:

Roberto Encarnacion Mosquera

Universidad del Cauca

Ingeniería electrónica y telecomunicaciones
Ingeniería de sistemas
Popayán, mayo

1. Introducción

En este informe se documenta la implementación de los patrones de diseño de comportamiento Template Method y State dentro del proyecto Sistema PiedraAzul, específicamente en el módulo principal de gestión de citas médicas.

La primera parte de la solución aplica el patrón Template Method para estructurar el proceso de validación de una cita médica. La clase abstracta define un flujo general de validación, mientras que las clases concretas especializan algunas reglas según el tipo de cita, como cita presencial o cita virtual.

La segunda parte aplica el patrón State para modelar el ciclo de vida de una consulta o cita médica. Este patrón permite que el comportamiento del objeto cambie según su estado actual, evitando el uso excesivo de condicionales y organizando las reglas de transición entre estados.

2. Objetivo

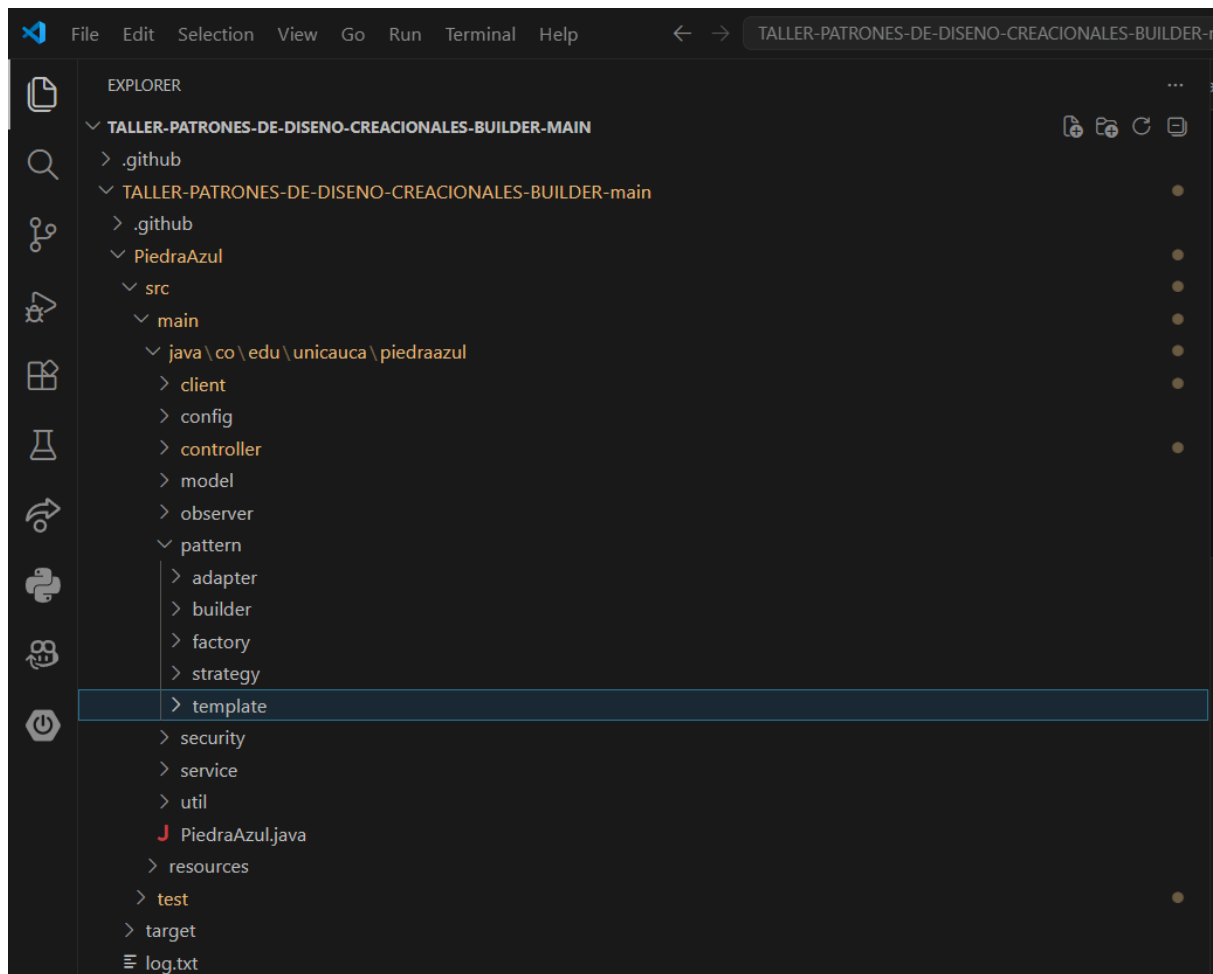
Implementar los patrones de diseño de comportamiento Template Method y State en el proyecto PiedraAzul, evidenciando el uso de clases abstractas, herencia, polimorfismo, separación de responsabilidades y ejecución funcional de los ejemplos aplicados al dominio de citas médicas.

3. Definiciones Principales

Concepto	Definición aplicada al proyecto
Patrón Template Method	Patrón de comportamiento que define la estructura general de un algoritmo en una clase abstracta, permitiendo que algunos pasos sean implementados o personalizados por las subclases.
Método plantilla	Método que establece el orden fijo de ejecución del algoritmo. En este proyecto corresponde al método validar() de la clase ValidadorCitaTemplate.
Clase abstracta	Clase que define el comportamiento general común y deja algunos pasos para que las clases hijas los implementen. En este caso corresponde a ValidadorCitaTemplate.
Clase concreta	Clase que hereda de la clase abstracta y personaliza una parte del algoritmo. En este proyecto corresponde a ValidadorCitaVirtual y ValidadorCitaPresencial.

Validación común	Conjunto de reglas compartidas por todos los tipos de cita, como validar paciente, médico, fecha y hora.
Validación específica	Regla particular que depende del tipo de cita. Por ejemplo, en una cita virtual se valida el correo electrónico y en una cita presencial se valida la observación inicial.
Patrón State	Patrón de comportamiento que permite que un objeto cambie su comportamiento dependiendo del estado interno en el que se encuentre.
Contexto	Clase principal que mantiene una referencia al estado actual y delega las acciones al estado correspondiente. En este proyecto será la clase Consulta.
Estado concreto	Clase que representa un estado específico de una consulta médica, como EstadoProgramada, EstadoConfirmada, EstadoEnAtencion, EstadoCompletada o EstadoCancelada.
Transición de estado	Cambio de un estado a otro como resultado de una acción válida. Por ejemplo, una consulta en estado Programada puede pasar a Confirmada.

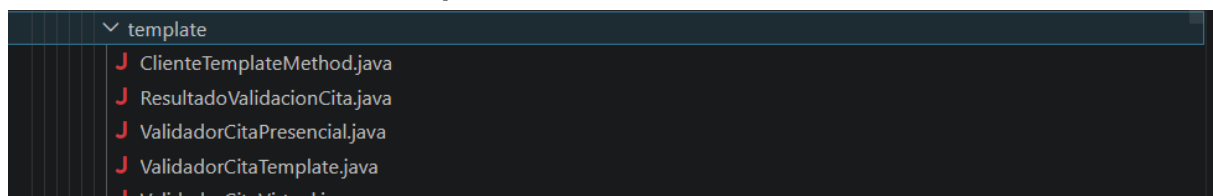
4. Estructura del Proyecto



La implementación se realizó dentro del módulo PiedraAzul. Para mantener una organización clara del código, se crearon dos paquetes nuevos dentro de pattern: el paquete template, donde se ubicaron las clases relacionadas con el patrón Template Method, y el paquete state, donde se ubicaron las clases correspondientes al patrón State.

Parte I: Patrón Template Method

5. Diseño de la solución Template Method

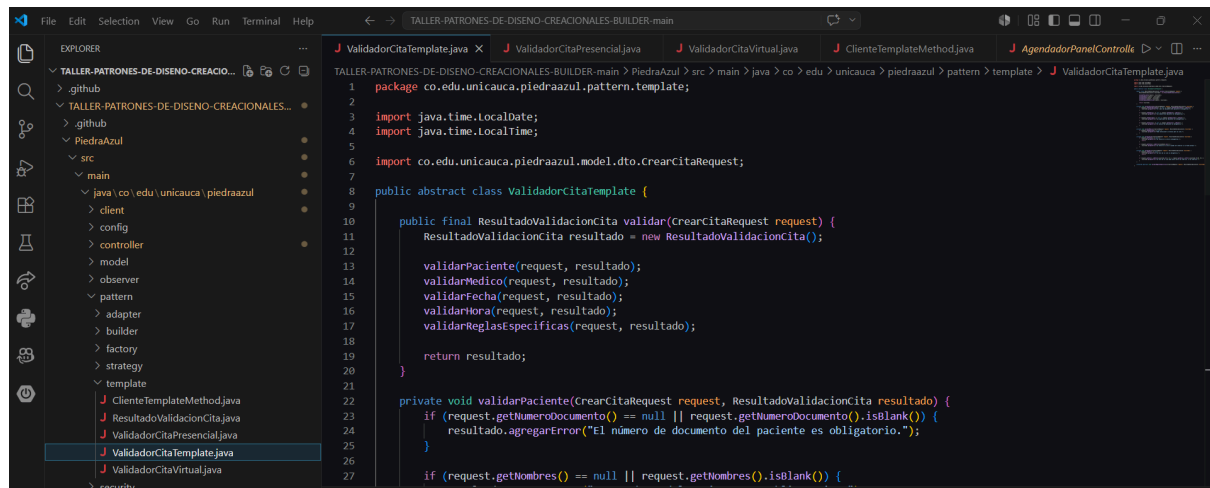


La solución aplica el patrón Template Method al proceso de validación de citas médicas. En este caso, todas las citas comparten una serie de validaciones comunes, como validar los datos del paciente, el médico, la fecha y la hora. Sin embargo, algunas reglas pueden variar dependiendo del tipo de cita.

Para resolver esto, se creó una clase abstracta llamada ValidadorCitaTemplate, la cual define el flujo general de validación mediante el método validar(). Este método

ejecuta las validaciones comunes y luego llama a un método abstracto llamado `validarReglasEspecificas()`, que debe ser implementado por las clases hijas. Las clases `ValidadorCitaVirtual` y `ValidadorCitaPresencial` heredan de `ValidadorCitaTemplate` y definen reglas particulares para cada tipo de cita. De esta forma, el algoritmo general permanece fijo, pero se permite especializar una parte del comportamiento.

6. Implementación de la clase abstracta

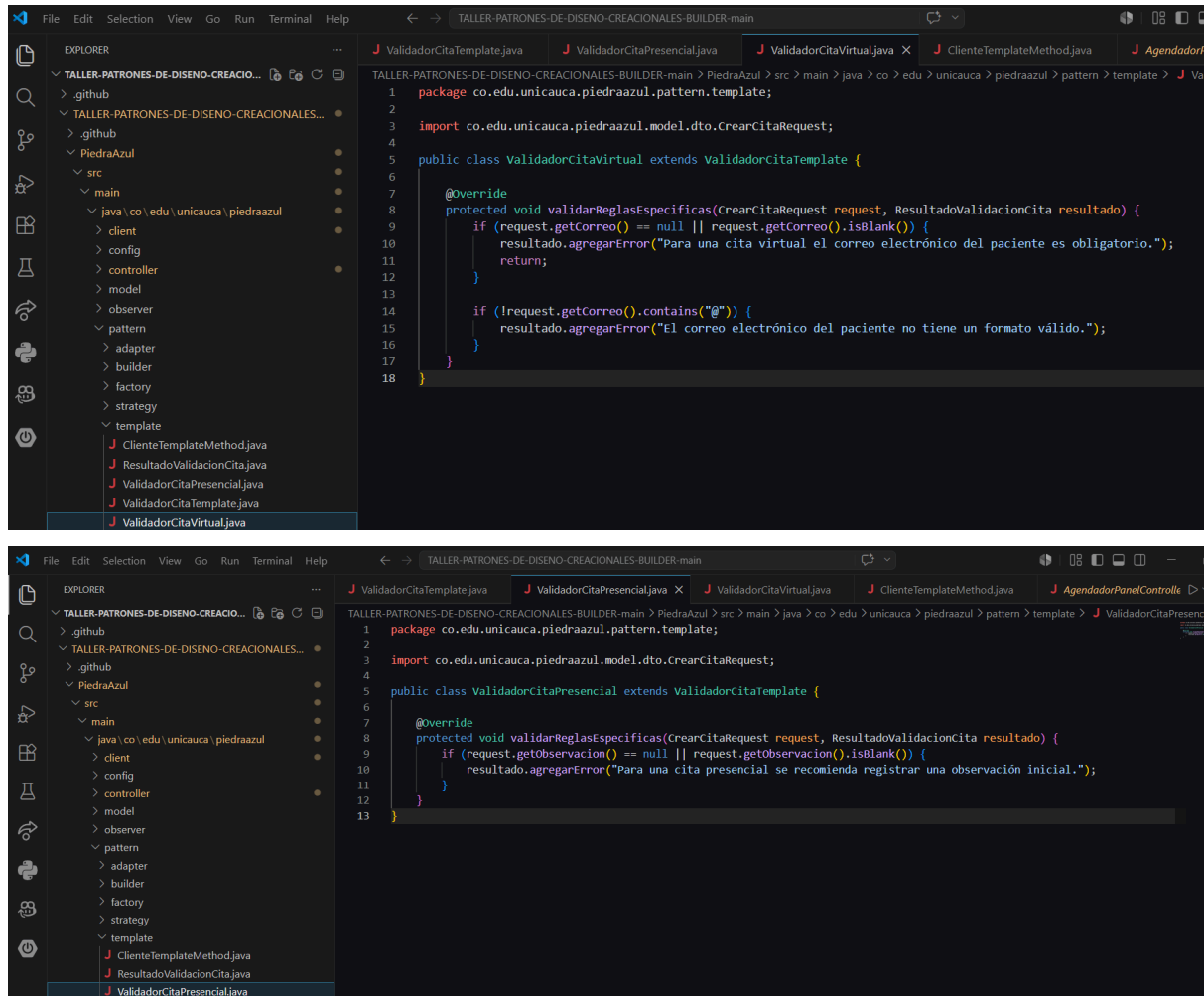


La clase **ValidadorCitaTemplate** representa la clase abstracta del patrón Template Method. Su responsabilidad principal es definir el flujo general de validación de una cita médica.

El método `validar()` corresponde al método plantilla, ya que establece el orden fijo en el que se ejecutan las validaciones. Este método no debe ser modificado por las subclases, por eso se declara como `final`.

Dentro del flujo se ejecutan validaciones comunes, como `validarPaciente()`, `validarMedico()`, `validarFecha()` y `validarHora()`. Finalmente, se invoca el método abstracto `validarReglasEspecificas()`, el cual permite que cada clase hija agregue sus propias reglas.

7. Implementación de las clases concretas

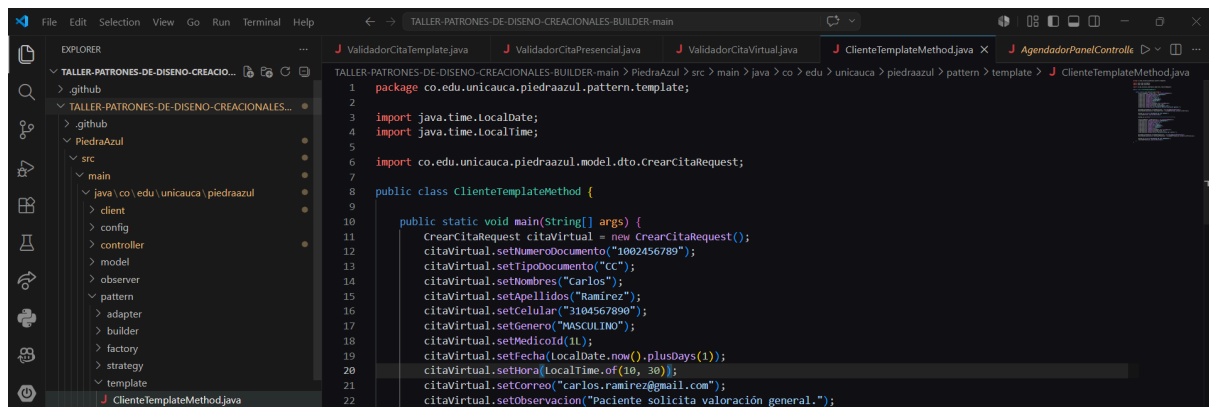


Las clases `ValidadorCitaVirtual` y `ValidadorCitaPresencial` representan las implementaciones concretas del patrón Template Method. Ambas clases heredan de `ValidadorCitaTemplate` y sobrescriben el método `validarReglasEspecificas()`.

ValidadorCitaVirtual agrega reglas relacionadas con las citas virtuales, como la obligatoriedad del correo electrónico del paciente y la validación básica de su formato.

ValidadorCitaPresencial agrega reglas propias de una cita presencial, como registrar una observación inicial. Con esto se demuestra que el flujo general de validación permanece centralizado en la clase abstracta, mientras que las variaciones se delegan a las clases concretas.

8. Cliente de prueba del Template Method



Para comprobar el funcionamiento del patrón Template Method se creó la clase **ClienteTemplateMethod**. Esta clase construye dos objetos **CrearCitaRequest**: uno correspondiente a una cita virtual y otro correspondiente a una cita presencial. Luego, cada objeto es validado mediante su respectivo validador concreto. El resultado de la validación se imprime en consola, demostrando que el método plantilla ejecuta correctamente el flujo definido y que cada subclase aplica sus reglas específicas.

9. Ejecución del patrón Template Method

```
[INFO] Total time: 6.933 s
[INFO] Finished at: 2026-05-17T11:56:58-05:00
[INFO] -----
PS C:\Users\Bryan\Desktop\TALLER-PATRONES-DE-DISENO-CREACIONALES-BUILDER-main\TALLER-PATRONES-DE-DISENO-CREACIONALES-BUILDER-main\PiedraAzul> java -cp target/classes co.edu.unicauca.piedraazul.pattern.template.ClienteTemplateMethod
VALIDACIÓN DE CITA VIRTUAL
La cita cumple con todas las validaciones.
-----
VALIDACIÓN DE CITA PRESENCIAL
La cita cumple con todas las validaciones.
```

La ejecución se realizó desde la terminal mediante la compilación del proyecto con Maven y la ejecución directa de la clase **ClienteTemplateMethod**. Como resultado, se evidencia que las citas virtual y presencial cumplen correctamente con las validaciones definidas.

Parte II: Patrón State

10. Diseño de la solución State

La solución aplica el patrón State al ciclo de vida de una consulta médica dentro del sistema **PiedraAzul**. Una consulta puede pasar por diferentes estados, como Programada, Confirmada, En Atención, Completada o Cancelada.

Cada estado tiene comportamientos diferentes frente a acciones como confirmar, iniciar atención, completar o cancelar. En lugar de manejar todas estas reglas mediante múltiples condicionales dentro de una sola clase, se creó una clase padre común llamada **EstadoConsulta** y varias clases concretas, una por cada estado. La clase **Consulta** actúa como contexto del patrón State. Esta clase mantiene el estado actual de la consulta y delega las acciones al objeto estado correspondiente.

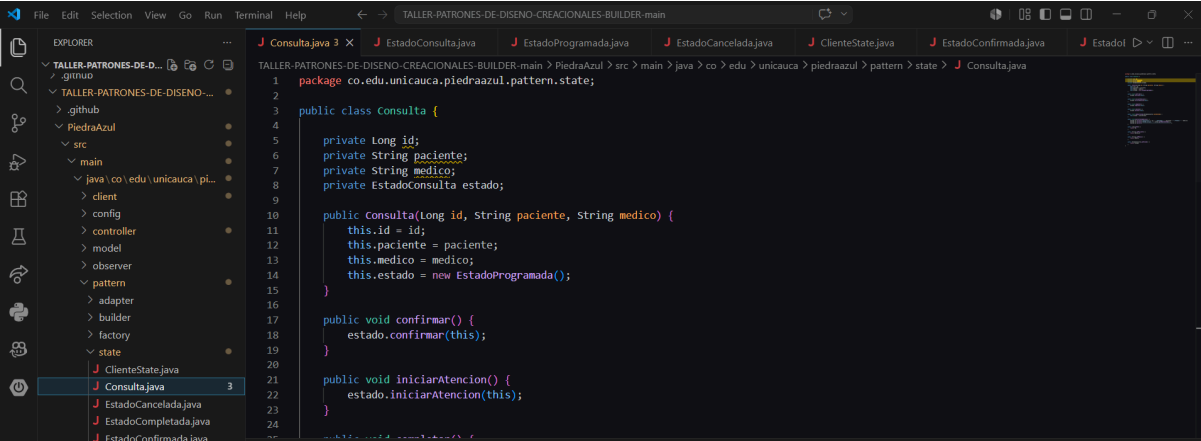
Así, el comportamiento cambia dinámicamente dependiendo del estado en el que se encuentre la consulta.

11. Tabla de transiciones de estado

Estado actual	Acciones permitidas	Acciones no permitidas
Programada	confirmar(), cancelar()	iniciarAtencion(), completar()
Confirmada	iniciarAtencion(), cancelar()	confirmar(), completar()
EnAtencion	completar()	confirmar(), iniciarAtencion(), cancelar()
Completada	Ninguna	confirmar(), iniciarAtencion(), completar(), cancelar()
Cancelada	Ninguna	confirmar(), iniciarAtencion(), completar(), cancelar()

La tabla anterior define las transiciones permitidas para una consulta médica. Cada estado controla qué acciones son válidas y cuáles no. Esto evita que la clase Consulta tenga que validar manualmente todas las combinaciones posibles mediante estructuras condicionales.

12. Implementación de la clase Consulta

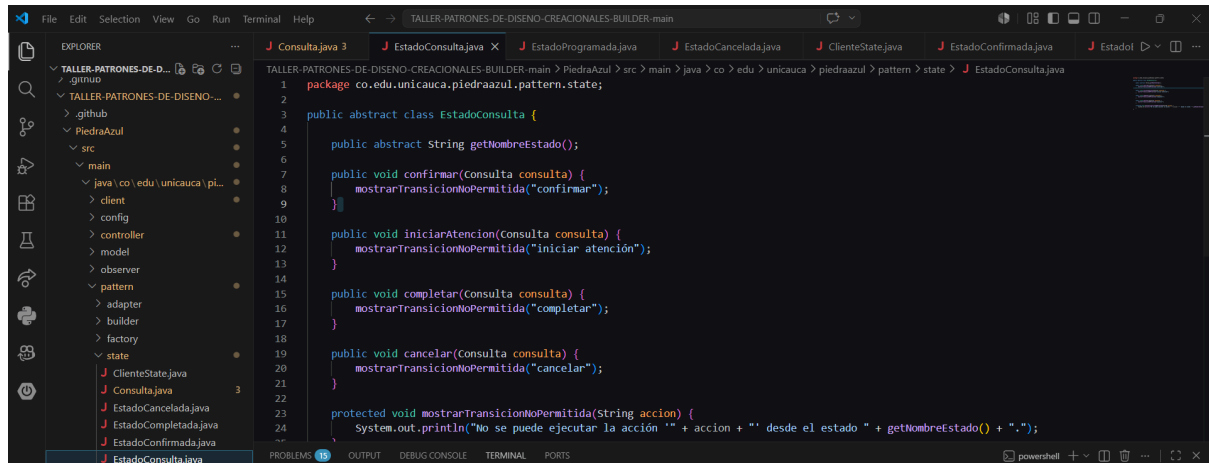


```
1 package co.edu.unicauca.piedraazul.pattern.state;
2
3 public class Consulta {
4
5     private Long id;
6     private String paciente;
7     private String medico;
8     private EstadoConsulta estado;
9
10    public Consulta(Long id, String paciente, String medico) {
11        this.id = id;
12        this.paciente = paciente;
13        this.medico = medico;
14        this.estado = new EstadoProgramada();
15    }
16
17    public void confirmar() {
18        estado.confirmar(this);
19    }
20
21    public void iniciarAtencion() {
22        estado.iniciarAtencion(this);
23    }
24
25    public void cancelar() {
26        estado.cancelar(this);
27    }
28
29    public void completar() {
30        estado.completar(this);
31    }
32
33    public Long getId() {
34        return id;
35    }
36
37    public String getPaciente() {
38        return paciente;
39    }
40
41    public String getMedico() {
42        return medico;
43    }
44
45    public EstadoConsulta getEstado() {
46        return estado;
47    }
48
49    public void setId(Long id) {
50        this.id = id;
51    }
52
53    public void setPaciente(String paciente) {
54        this.paciente = paciente;
55    }
56
57    public void setMedico(String medico) {
58        this.medico = medico;
59    }
60
61    public void setEstado(EstadoConsulta estado) {
62        this.estado = estado;
63    }
64
65    @Override
66    public String toString() {
67        return "Consulta{" +
68            "id=" + id +
69            ", paciente='" + paciente + '\'' +
70            ", medico='" + medico + '\'' +
71            ", estado=" + estado +
72            '}';
73    }
74
75 }
```

La clase Consulta corresponde al contexto del patrón State. Esta clase contiene la información básica de la consulta y mantiene una referencia al estado actual mediante el atributo estado.

Cuando se ejecutan acciones como confirmar(), iniciarAtencion(), completar() o cancelar(), la clase Consulta no resuelve directamente la lógica, sino que delega el comportamiento al estado actual. Esto permite que cada estado decida cómo responder ante cada acción.

13. Implementación de la clase padre EstadoConsulta

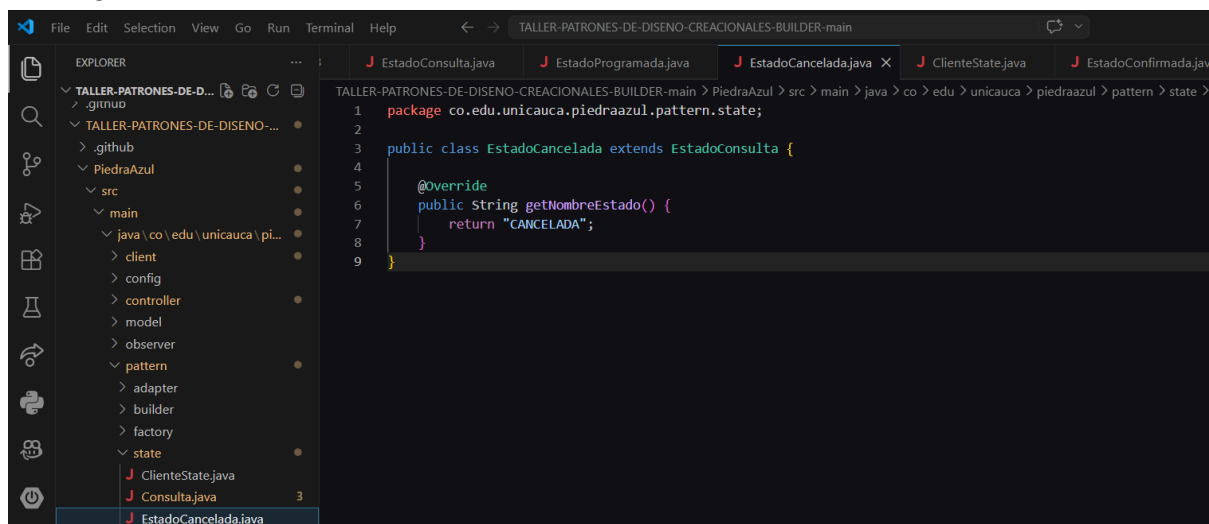


```
1 package co.edu.unicauca.piedraazul.pattern.state;
2
3 public abstract class EstadoConsulta {
4
5     public abstract String getNombreEstado();
6
7     public void confirmar(Consulta consulta) {
8         mostrarTransicionNoPermitida("confirmar");
9     }
10
11     public void iniciarAtencion(Consulta consulta) {
12         mostrarTransicionNoPermitida("iniciar atención");
13     }
14
15     public void completar(Consulta consulta) {
16         mostrarTransicionNoPermitida("completar");
17     }
18
19     public void cancelar(Consulta consulta) {
20         mostrarTransicionNoPermitida("cancelar");
21     }
22
23     protected void mostrarTransicionNoPermitida(String accion) {
24         System.out.println("No se puede ejecutar la acción '" + accion + "' desde el estado " + getNombreEstado() + ".");
25     }
26 }
```

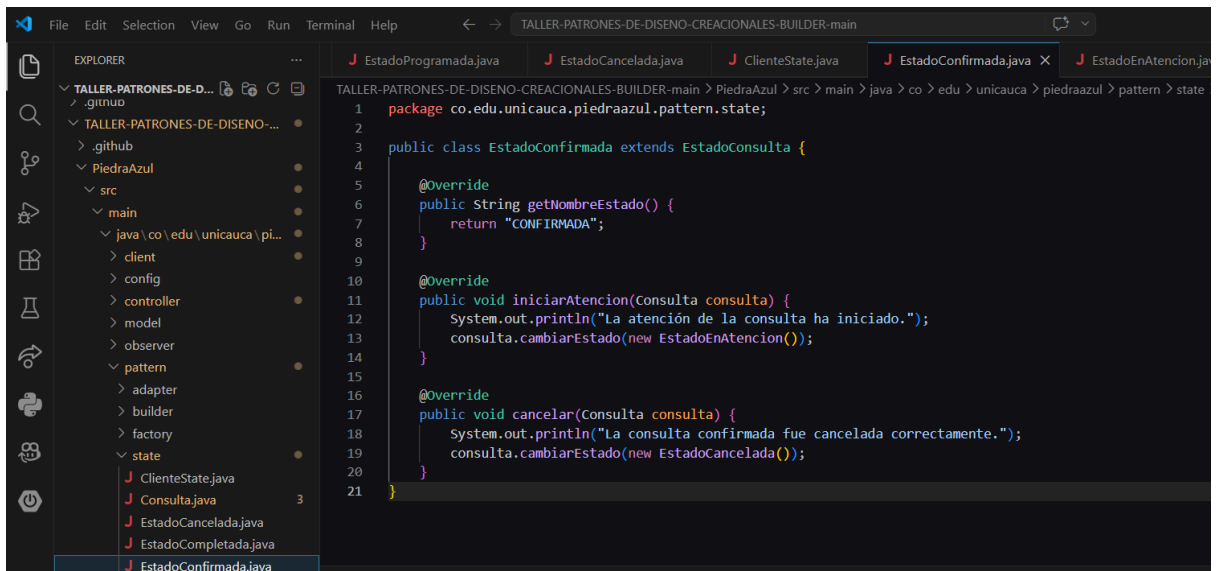
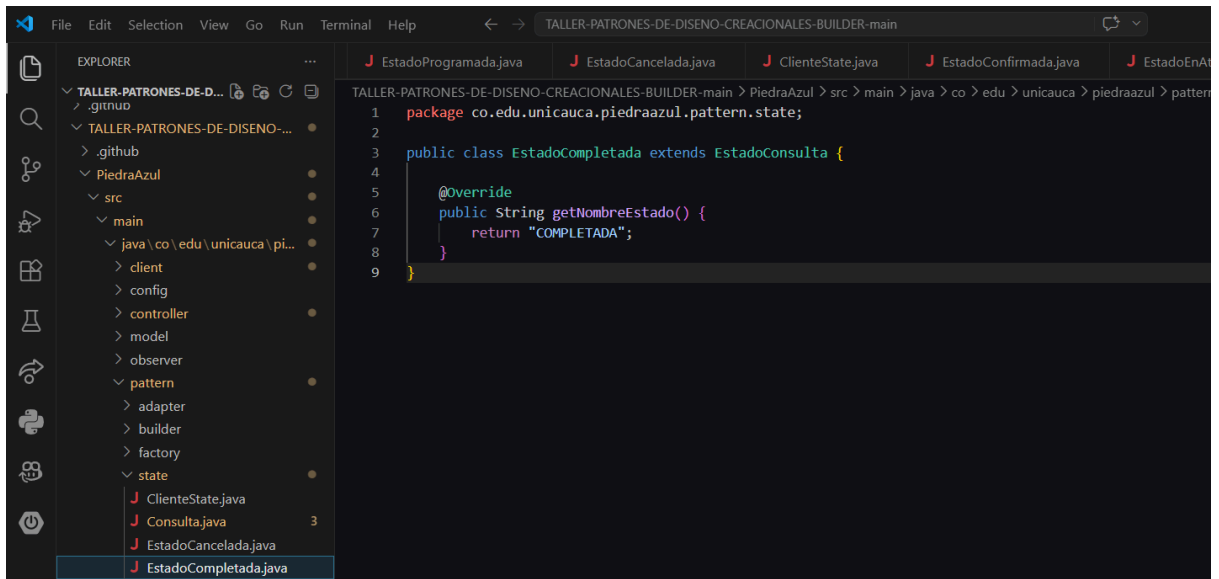
La clase EstadoConsulta representa la clase padre común para todos los estados de una consulta. En ella se declaran los métodos que representan las acciones posibles del ciclo de vida de la consulta.

Cada estado concreto hereda de esta clase y redefine únicamente las acciones que son válidas desde ese estado. Las acciones no permitidas pueden mostrar un mensaje indicando que la transición no es válida.

14. Implementación de los estados concretos



```
1 package co.edu.unicauca.piedraazul.pattern.state;
2
3 public class EstadoCancelada extends EstadoConsulta {
4
5     @Override
6     public String getNombreEstado() {
7         return "CANCELADA";
8     }
9 }
```



The screenshot shows an IDE with the Explorer panel on the left displaying the project structure. The main editor shows the implementation of the `EstadoEnAtencion` class, which extends `EstadoConsulta`. The class has two methods: `getNombreEstado()` returning "EN_ATENCION" and `completar(Consulta consulta)` which prints a completion message and transitions to `EstadoCompletada`.

```
2  
3  
4 public class EstadoEnAtencion extends EstadoConsulta {  
5  
6     @Override  
7     public String getNombreEstado() {  
8         return "EN_ATENCION";  
9     }  
10  
11     @Override  
12     public void completar(Consulta consulta) {  
13         System.out.println("La consulta fue completada correctamente.");  
14         consulta.cambiarEstado(new EstadoCompletada());  
15     }  
16 }
```

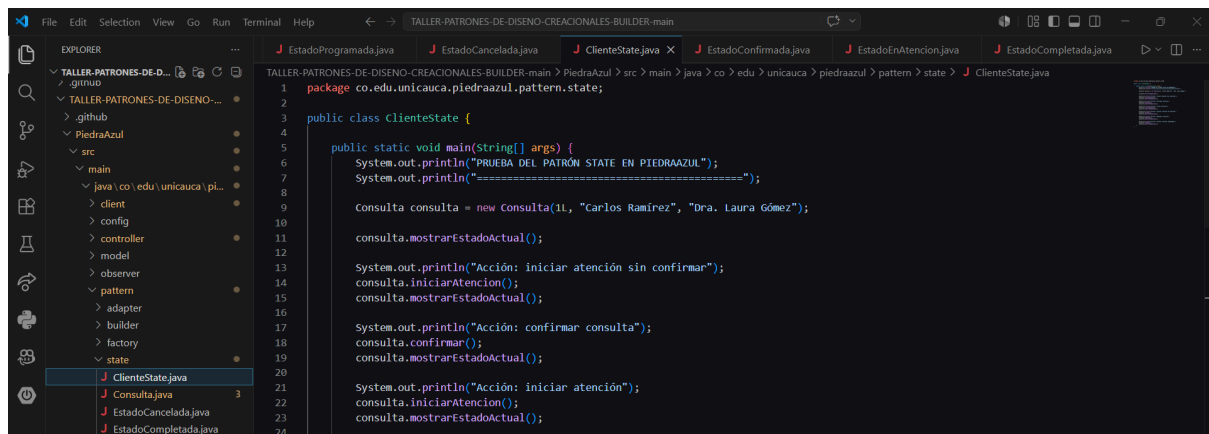
The screenshot shows the IDE with the Explorer panel on the left. The main editor shows the implementation of the `EstadoProgramada` class, which extends `EstadoConsulta`. The class has three methods: `getNombreEstado()` returning "PROGRAMADA", `confirmar(Consulta consulta)` which prints a confirmation message and transitions to `EstadoConfirmada`, and `cancelar(Consulta consulta)` which prints a cancellation message and transitions to `EstadoCancelada`.

```
1 package co.edu.unicauca.piedraazul.pattern.state;  
2  
3 public class EstadoProgramada extends EstadoConsulta {  
4  
5     @Override  
6     public String getNombreEstado() {  
7         return "PROGRAMADA";  
8     }  
9  
10     @Override  
11     public void confirmar(Consulta consulta) {  
12         System.out.println("La consulta fue confirmada correctamente.");  
13         consulta.cambiarEstado(new EstadoConfirmada());  
14     }  
15  
16     @Override  
17     public void cancelar(Consulta consulta) {  
18         System.out.println("La consulta programada fue cancelada correctamente.");  
19         consulta.cambiarEstado(new EstadoCancelada());  
20     }  
21 }
```

Los estados concretos representan cada etapa del ciclo de vida de una consulta médica. Cada clase define su propio comportamiento frente a las acciones disponibles.

EstadoProgramada permite confirmar o cancelar la consulta. **EstadoConfirmada** permite iniciar la atención o cancelar. **EstadoEnAtencion** permite completar la consulta. **EstadoCompletada** y **EstadoCancelada** representan estados finales, por lo tanto no permiten nuevas transiciones.

15. Cliente de prueba del patrón State



Para validar el funcionamiento del patrón State se creó la clase **ClienteState**. Esta clase instancia una consulta en estado Programada y ejecuta una secuencia de acciones permitidas: confirmar, iniciar atención y completar.

También se pueden probar acciones no permitidas, como intentar cancelar una consulta cuando ya se encuentra completada. De esta forma se evidencia que el comportamiento de la consulta depende del estado actual.

16. Ejecución del patrón State

```
PS C:\Users\Brayan\Desktop\TALLER-PATRONES-DE-DISENO-CREACIONALES-BUILDER-main\TALLER-PATRONES-DE-DISENO-CREACIONALES-BUILDER-main\PiedraAzul> java -cp target/classes co.edu.unicauca.piedraazul.pattern.state.ClienteState
PRUEBA DEL PATRÓN STATE EN PIEDRAAZUL
-----
Consulta #1 | Paciente: Carlos Ramírez | Médico: Dra. Laura Gómez
Estado actual: PROGRAMADA
-----
Acción: iniciar atención sin confirmar
No se puede ejecutar la acción 'iniciar atención' desde el estado PROGRAMADA.
Consulta #1 | Paciente: Carlos Ramírez | Médico: Dra. Laura Gómez
Estado actual: PROGRAMADA
-----
Acción: confirmar consulta
La consulta fue confirmada correctamente.
Consulta #1 | Paciente: Carlos Ramírez | Médico: Dra. Laura Gómez
Estado actual: CONFIRMADA
-----
Acción: iniciar atención
La atención de la consulta ha iniciado.
Consulta #1 | Paciente: Carlos Ramírez | Médico: Dra. Laura Gómez
Estado actual: EN_ATENCION
-----
Acción: cancelar consulta en atención
No se puede ejecutar la acción 'cancelar' desde el estado EN_ATENCION.
Consulta #1 | Paciente: Carlos Ramírez | Médico: Dra. Laura Gómez
Estado actual: EN_ATENCION
-----
Acción: completar consulta
La consulta fue completada correctamente.
Consulta #1 | Paciente: Carlos Ramírez | Médico: Dra. Laura Gómez
Estado actual: COMPLETADA
-----
Acción: cancelar consulta completada
No se puede ejecutar la acción 'cancelar' desde el estado COMPLETADA.
Consulta #1 | Paciente: Carlos Ramírez | Médico: Dra. Laura Gómez
Estado actual: COMPLETADA
-----
```

La ejecución del patrón State permitió evidenciar que la consulta cambia correctamente de estado según las acciones realizadas. Además, se comprobó que las transiciones no permitidas son rechazadas por el estado correspondiente, demostrando que el comportamiento fue distribuido entre las clases de estado y no concentrado en la clase Consulta.

17. Conclusiones

Con el desarrollo de este taller se logró implementar los patrones de diseño de comportamiento Template Method y State dentro del proyecto PiedraAzul. La implementación permitió mejorar la organización del código y aplicar conceptos fundamentales de la programación orientada a objetos como abstracción, herencia, polimorfismo y separación de responsabilidades.

El patrón Template Method fue aplicado al proceso de validación de citas médicas. La clase ValidadorCitaTemplate permitió definir un flujo general de validación, mientras que las clases ValidadorCitaVirtual y ValidadorCitaPresencial permitieron especializar reglas concretas sin modificar el algoritmo principal. Esto demuestra una de las ventajas más importantes del patrón: mantener estable la estructura del proceso y permitir variaciones controladas mediante subclases.

Por otro lado, el patrón State fue aplicado al ciclo de vida de una consulta médica. La clase Consulta delega su comportamiento al estado actual, representado por clases como EstadoProgramada, EstadoConfirmada, EstadoEnAtencion, EstadoCompletada y EstadoCancelada. Esta solución evita concentrar múltiples condicionales dentro de una sola clase y permite que cada estado tenga una responsabilidad clara.

En conjunto, ambos patrones fortalecen el diseño del sistema PiedraAzul. Template Method organiza procesos que siguen una estructura común, mientras que State permite modelar objetos cuyo comportamiento depende de su estado interno. La implementación realizada facilita el mantenimiento, mejora la claridad del código y permite extender el sistema en el futuro con nuevos tipos de validaciones o nuevos estados de consulta sin afectar significativamente las clases existentes.